Градиентный спуск 🪄✨

Лекция 6

Это просто великолепная штука!

Применяется везде, где надо искать какой-то максимум или минимум:
в ИИ, экономике, рекомендательных системах, предсказание болезней, калибровке камер Теслы, оптимизации маршрутов и прочее.

Но мы подберёмся к нему издалека…

Second partial derivative test — скорее, теоретический метод. Его преимущество в том, что он находит точные решения и сразу все экстремумы, если их несколько. В реальных задачах осуществление этого теста может быть трудоёмким, особенно в многомерных пространствах или при работе с непростыми функциями. В целом, стать плохо может даже на полиномах:

$$z = (x-1)^2(y-2)^4 + (x-1)^4(y-2)^2 + (x-1)^2(y-2)^2$$

$$\frac{\partial z}{\partial x} = 2(x-1)(y+1)^4 + 4(x+1)^3(y-1)^2 + 2(x-3)(y-2)^2;$$

$$\frac{\partial z}{\partial y} = 4(x-1)^2(y+1)^3 + 2(x+1)^4(y-1) + 2(x-3)^2(y-2).$$

Если приравнять их к нулю, то получится нелинейная система уравнений. Для её решения нужны сложные методы, которые требуют много времени и вычислительных ресурсов (например, высокой мощности процессора).

А ведь это всего лишь полиномиальная функция, тут нет ни логарифмов, ни экспонент, ни тригонометрических функций! На практике бывают и более сложные системы, для решения которых нет готовых эффективных программных реализаций.

И именно в таких случаях на помощь приходит ОН…

Введём несколько определений.

Пусть дана непрерывная ФНП. Если мы находимся в какой-то её точке, то можно пойти в разных направлениях и посмотреть, как быстро и сильно меняется знач. функции:

- по оси x изменение задаёт частная производная f'_x
- по оси y — производная f'_y

Но это же **не единственные направления** (линии), вдоль которых можно гулять на плоскости XOY, у нас на прошлом занятии была функция, где седловидность была хорошо видна вдоль прямой y=x
https://www.math3d.org/VYULYKbMAS

Рассмотрим движение от точки (x, y) по направлению к точке $(x + \Delta x, y + \Delta y)$. Направление задаёт вектор $\vec{u} = (\Delta x, \Delta y)$. Нас интересует лишь направление, поэтому давайте рассматривать только векторы единичной длины: примем, что длина вектора $\vec{u}$ равна 1, то есть $\sqrt{(\Delta x)^2 + (\Delta y)^2} = 1$.

Производная по направлению — это одно из обобщений понятия производной на случай нескольких переменных. Производная по направлению показывает, как быстро значение функции изменяется при движении в направлении заданного вектора.

Мы будем пользоваться такой формулой:

$$f'_{\vec{u}} = \Delta x \cdot \frac{\partial f}{\partial x} + \Delta y \cdot \frac{\partial f}{\partial y}.$$

Геом. смысл производной по направлению: строим плоскость, проходящую через эту линию направления и перпендикулярно xOy, там строим касательную прямую к поверхности, смотрим угол между касат. и линией направления.

$$f'_{\vec{u}} = \Delta x \cdot \frac{\partial f}{\partial x} + \Delta y \cdot \frac{\partial f}{\partial y}.$$

Если вы уже изучали линал, то, возможно, эта формула вам кого-то напоминает… сумма произведений…

$$f'_{\vec{u}} = \Delta x \cdot \frac{\partial f}{\partial x} + \Delta y \cdot \frac{\partial f}{\partial y}.$$

Если вы уже изучали линал, то, возможно, эта формула вам кого-то напоминает… сумма произведений…

Это формула для скалярного произведения двух векторов!

Получается, что $f'_{\vec{u}} = (\Delta x, \Delta y) \cdot \left(\frac{\partial f}{\partial x}, \frac{\partial f}{\partial y} \right)$.

Здесь вектор $(\Delta x, \Delta y)$ — это знакомый нам $\vec{u}$, а второй множитель имеет особое название.

Градиент функции f — вектор, состоящий из частных производных: $\left(\frac{\partial f}{\partial x}, \frac{\partial f}{\partial y} \right)$.

Обозначается как ∇f.

Чтобы найти градиент функции в точке (x_0, y_0), нужно рассчитать в ней значения частных производных:

$$\nabla f = \left(\frac{\partial f}{\partial x}(x_0, y_0), \frac{\partial f}{\partial y}(x_0, y_0) \right).$$

Ещё есть обозначение grad (f)

И снова: ничего не напоминает нам эта история с частными производными?

Чтобы найти градиент функции в точке (x_0, y_0), нужно рассчитать в ней значения частных производных:

$$\nabla f = \left(\frac{\partial f}{\partial x}(x_0, y_0), \frac{\partial f}{\partial y}(x_0, y_0) \right).$$

Ещё есть обозначение grad (f)

И снова: ничего не напоминает нам эта история с частными производными?

Градиент — это частный случай матрицы Якоби.
Матрица Якоби — это матрица всех частных производных вектор-функции, а градиент — это матрица (вектор) скалярной функции (то есть обычной, которая выдаёт на выходе только одно число)

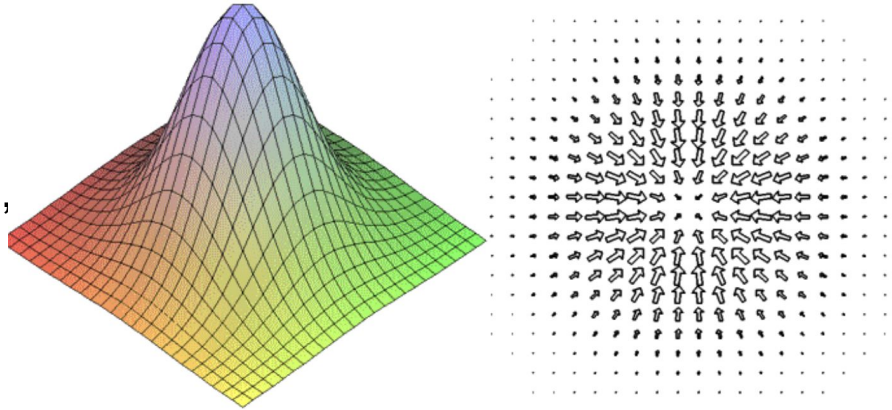
$$J_{\mathbf{F}} = \begin{pmatrix} \frac{\partial f_1}{\partial x_1} & \cdots & \frac{\partial f_1}{\partial x_n} \\ \vdots & \ddots & \vdots \\ \frac{\partial f_m}{\partial x_1} & \cdots & \frac{\partial f_m}{\partial x_n} \end{pmatrix} \qquad \nabla f = \begin{pmatrix} \frac{\partial f}{\partial x_1} \\ \vdots \\ \frac{\partial f}{\partial x_n} \end{pmatrix}$$

У градиента столько же координат, сколько аргументов у функции. У нас функция двух переменных, поэтому у градиента две координаты.

И вот ВНЕЗАПНО (на самом деле не очень) оказывается, что направление **наискорейшего роста** функции — как раз **по градиенту.**
Направление, по которому функция быстрее всего убывает, — противоположное ему, вдоль вектора **антиградиента**.

Специального обозначения для него нет, его записывают просто как «$-\nabla f$».

Мы не будем это доказывать (некоторые вообще вшивают это прям в определение), но будем активно этиим пользоваться!



Вот отличное видео про это.

Вычислим градиент в точке (1, π) для функции $f(x, y) = x^2 y + \sin(xy)$

Вычислим градиент в точке (1, π) для функции $f(x, y) = x^2 y + \sin(xy)$

$$f'_x = 2xy + \cos(xy) \cdot y$$

$$f'_y = x^2 + \cos(xy) \cdot x$$

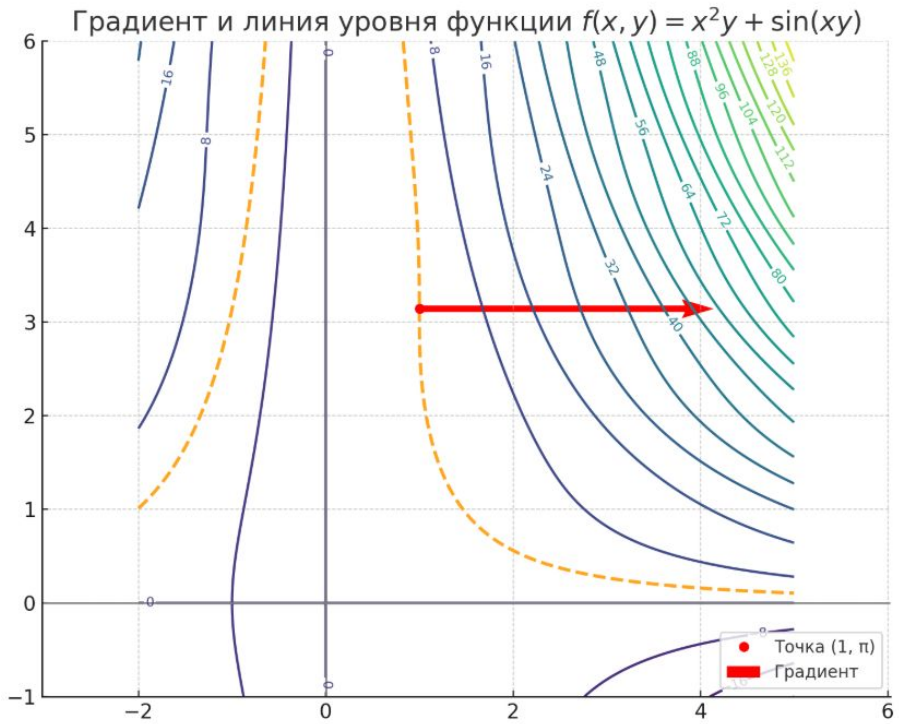
Значит, $\vec{\nabla} f(x, y) = \left(2xy + y\cos(xy),\ x^2 + x\cos(xy)\right)$

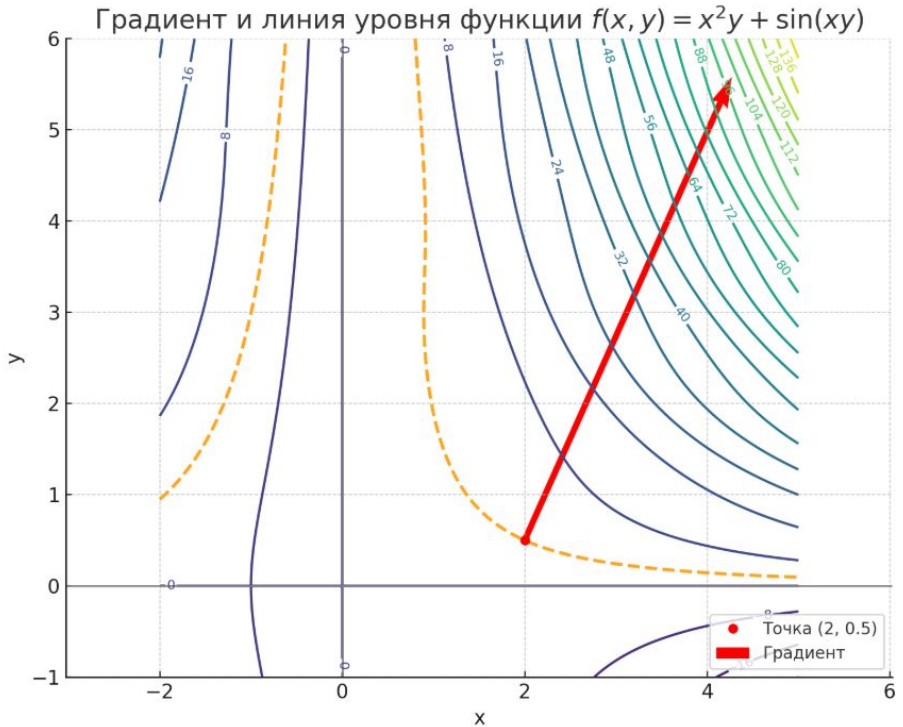
И подставим точку (1, π):

$$\vec{\nabla} f(1, \pi) = \left(2 \cdot 1 \cdot \pi + \pi\cos(\pi),\ 1^2 + 1 \cdot \cos(\pi)\right) = (2\pi - \pi,\ 1 - 1) = (\pi,\ 0)$$

Такой результат означает, что в этой точке функция быстрее всего возрастает в сторону роста оси икс

Интересный момент: вектор градиента в точке всегда перпендикулярен линии уровня в этой точке.





Как оценивать перпендикулярность кривой и вектора: надо провести касательную к кривой в этой точке и мерить угол между ней и вектором.

Значит, следование вектору градиента поможет нам найти локальный максимум, а следование вектору антиградиента — локальный минимум. И это может оказаться сильно лучше, чем по необх/дост условию экстремума.

Если мы хотим считать это не руками, то имеет смысл сузить область поиска (щас решением может оказаться кто угодно из обл. определения).
А ещё откажемся от идеи найти сразу все экстремумы — чаще всего в реальных задачах нам нужны только определённые их типы. В обучении нейросетей принято рассматривать задачи на минимум. Поэтому будем искать только минимумы.

Интуиция такая: встанем в рандомную точку и будем шагать против градиента, каждый раз корректируя направление.

А теперь формально!

Градиентный спуск

Это название того самого алгоритма поиска точки локального экстремума (и самого экстремума) с помощью градиента. По умолчанию он используется для нахождения минимума, но можно перефразировать и для максимума.

Алгоритм нахождения точек минимума функции с помощью градиентного спуска:

1) Выбрать начальную точку.

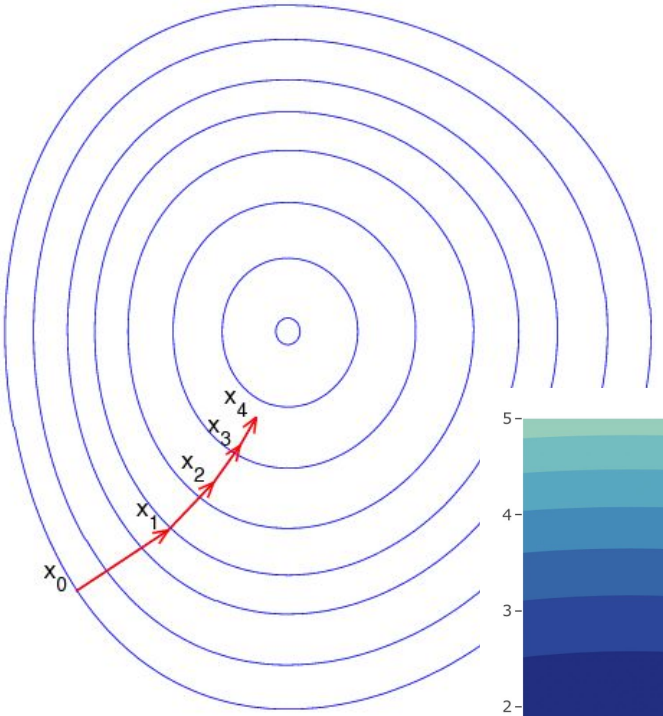
2) Сдвинуться из этой точки против направления градиента. Получить новую точку.

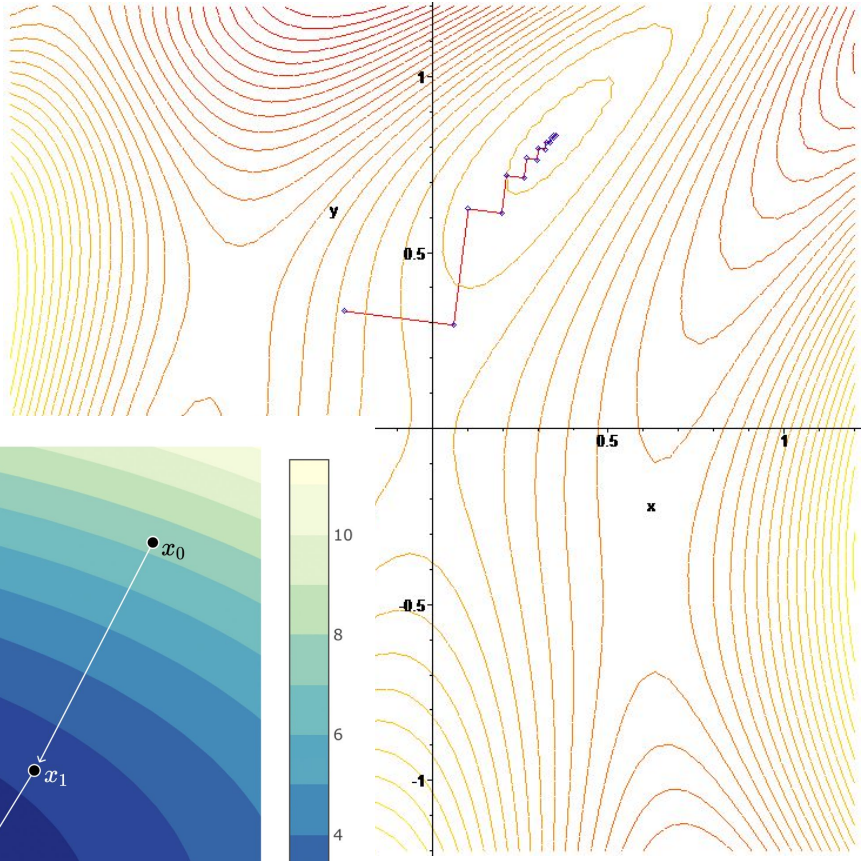
3) Повторять второй шаг, пока не подойдём к минимуму достаточно близко.

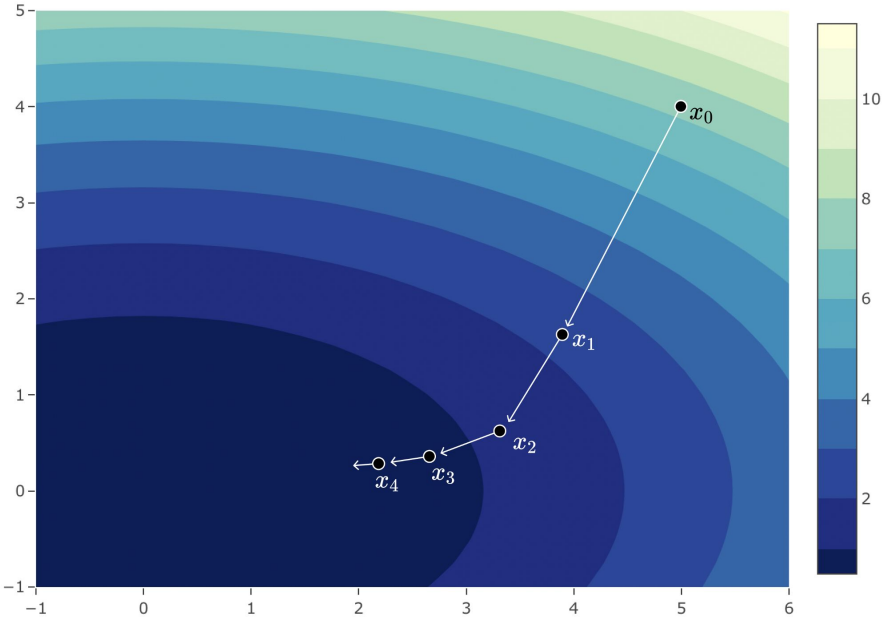
Начальную точку обычно выбирают случайно. Иногда по задаче понятно, какую точку лучше взять начальной.

Координаты новой точки через координаты предыдущей вычисляются по формуле:

$$(x_{\text{new}}, y_{\text{new}}) = (x_{\text{old}}, y_{\text{old}}) - \nabla f(x_{\text{old}}, y_{\text{old}}).$$


x0
x1
x2
x3
x4


1
0.5
y
0.5
1
x
-0.5
-1


5
4
3
2
1
0
−1
−1
0
1
2
3
4
5
6
x_0
x_1
x_2
x_3
x_4
10
8
6
4
2

Получается, что из начальной точки мы как бы **спускаемся против градиента** всё ниже и ниже к минимуму. Поэтому метод и называется градиентным спуском. А сдвиг к следующей точке называется **шагом спуска.**

Это почти как спускаться с горы в темноте маленькими аккуратными шажочками.

Как выбираем начальную точку

Случайно. :) Или по контексту задачи можно что-то нащупать близкое к минимуму и стартовать оттуда. Но об этом вам подробно расскажут в кодерской части.

Как шагаем

Берём точку и откладываем от неё вектор антиградиента. Либо вычесть из её координат координаты обычного градиента для этой исходной точки.

Координаты новой точки = координаты старой – градиент в старой точке

Рассчитаем 2 шага алгоритма для поиска минимума функции

$p(x, y) = x^4 + 0.5y^4 - 2x + 3y$. За начальную точку выберем $a_0 = (1, 1)$.

Найдём градиент функции:

$$\frac{\partial p}{\partial x}(x, y) = 4x^3 - 2;$$

$$\frac{\partial p}{\partial y}(x, y) = 2y^3 + 3;$$

$$\nabla p(x, y) = (4x^3 - 2, 2y^3 + 3).$$

Рассчитаем значение градиента в начальной точке:

$$\nabla p(1, 1) = (4 \cdot 1^3 - 2, 2 \cdot 1^3 + 3) = (2, 5).$$

Вычтем из начальной точки координаты градиента, чтобы получить следующую точку:

$$a_1 = a_0 - \nabla p(x_0, y_0) = (1, 1) - (2, 5) = (-1, -4).$$

Для второго шага нам нужна точка $a_1 = (-1, -4)$ и значение градиента в ней:

$$\nabla p(-1, -4) = (4 \cdot (-1)^3 - 2, 2 \cdot (-4)^3 + 3) = (-6, -125).$$

$$a_2 = a_1 - \nabla p(x_1, y_1) = (-1, -4) - (-6, -125) = (5, 121).$$

Только вот незадача: минимум на самом деле в районе точки (0,7937, -1,1447), а мы к ней даже не приблизились за два шага.

Может, шаг был слишком большой или с начальной точкой не повезло? Мы пошли в нужном направлении, но шагнули слишком далеко. Поэтому оказались в точке по другую сторону от минимума. Значение функции в ней даже больше, чем было в начальной.

Вывод: нам нужно делать шаги короче. Но как?

Будем домножать вычитаемое на какое-то маленькое число.

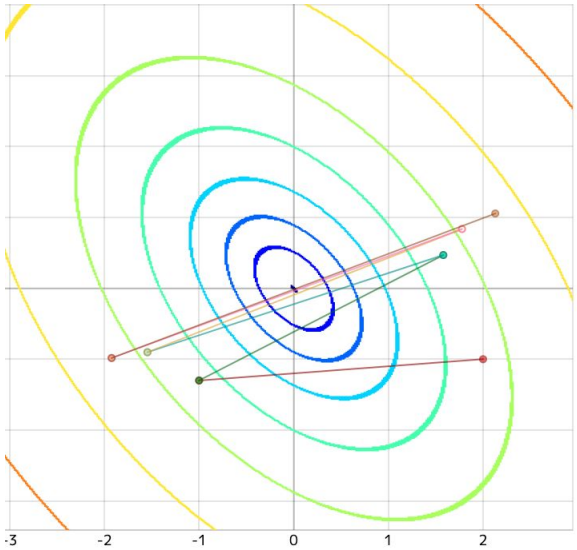
$$x^k = x^{k-1} - \nabla f(x^{k-1}) \cdot \gamma$$

Этот коэффициент имеет разные названия: «коэффициент обучения», «скорость обучения» (по-английски — “learning rate”) и другие. Часто встречается название «размер шага» или «длина шага». Формально это неверно, ведь фактический размер шага равен произведению этого коэффициента и градиента функции. Можно встретить обозначения разными буквами, обычно это гамма, альфа или эта: α, γ, η.

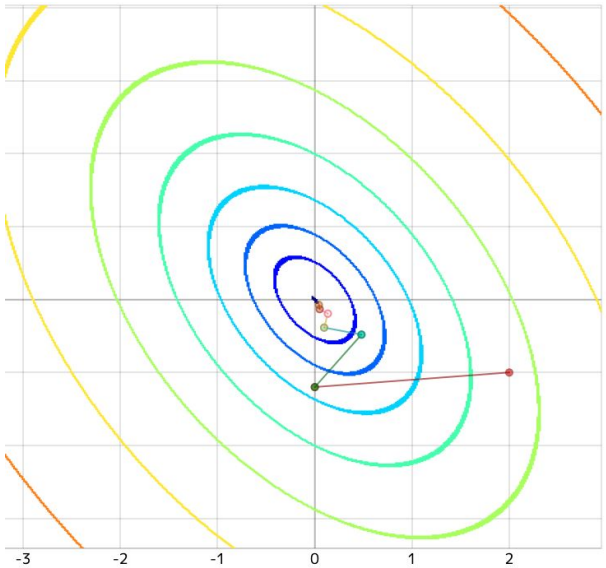
На практике его подбирают отдельно под каждую функцию, учитывая два момента:
- если шаг слишком большой, то алгоритм может никогда не прийти к минимуму;
- если шаг слишком маленький, то алгоритм может сходиться к минимуму очень долго.

$$x^k = x^{k-1} - \nabla f(x^{k-1}) \cdot \gamma$$

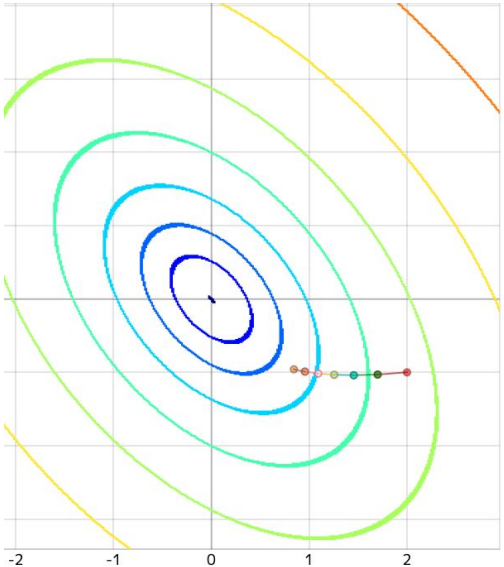
γ=3



γ=2



γ=0.3



Вот тут можно поиграться с начальной точкой и скоростью обучения:
https://elc.github.io/gradient-descent-method/

То есть нужно найти такой коэффициент, при котором значение функции уменьшается, причём довольно быстро. К сожалению, нет универсального правила, которое всегда даёт наилучший результат. Но есть разные рецепты, которые можно использовать. Один из них — перебирать отрицательные степени десятки. Популярные стартовые значения: $10^{-3}, 10^{-2}, 10^{-1}$.

С шагом разобрались, а когда останавливаться? Есть разные критерии остановки:
- значение в новой точке почти не отличается от значения в старой (изменение значения функции на последнем шаге меньше заданной точности),
- функция достигает заранее выбранного маленького значения,
- количество итераций достигло максимально заданного,
- превышен лимит времени работы программы.

Выбор критерия зависит от конкретной задачи и вычислительных мощностей. За приближённое значение точки минимума берутся координаты последней найденной точки.

Преимущества и недостатки градиентного спуска

Минусы: он ищет только локальный (а не глобальный) минимум, а также часто сильно зависит от нач. точки. Но во многих задачах он работает быстрее D-теста. Также есть «апгрейды», которые делают его ещё более эффективным.

Градиентный спуск особенно хорош в случаях, когда

- Размерность велика: у теста могут случиться проблемы уже при 3-4 переменных, а градиентным спуском решают задачи с сотнями неизвестных!
- Функция сложна: например, сложно дифференцируемые, не везде дифференцируемые и даже функции, к которым сложно подобрать формулу. Найти производную в таких случаях сложно или невозможно. А ГС помогает подойти к точке экстремума достаточно близко.
- Данные неточны: реальные данные часто содержат ошибки, и в таких случаях тест может привести к неоптимальным решениям. ГС обрабатывает даже «зашумлённые» данные и подбирается к хорошему решению.

Бонусы

- Тестовые функции для проверки вашего градиентного спуска https://ru.wikipedia.org/wiki/Тестовые_функции_для_оптимизации

- Учебник по ML от я.образования, там есть много про градиентный спуск во второй главе (и позже тоже) https://education.yandex.ru/handbook/ml

- Ещё штучки по теме https://github.com/j-w-yun/optimizer-visualization

На сегодня всё!
Вопросики?